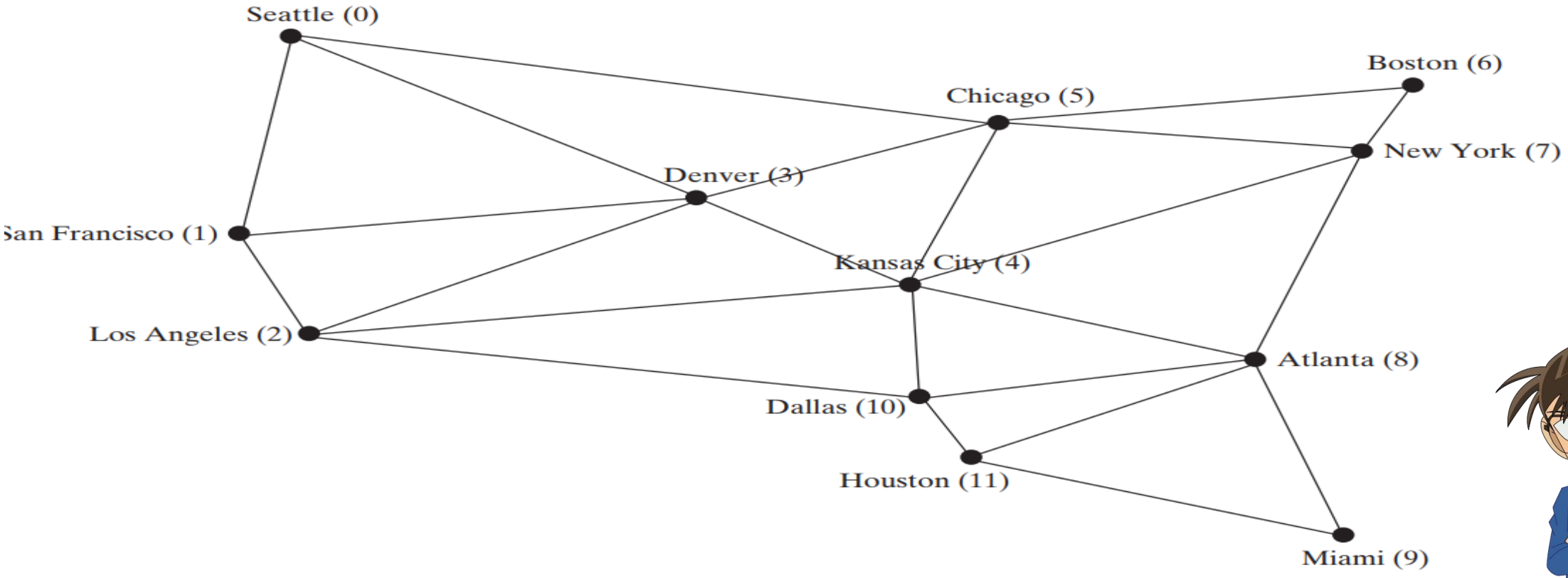
Graph & Its Application

Faculty of Information Technology, Hanoi University

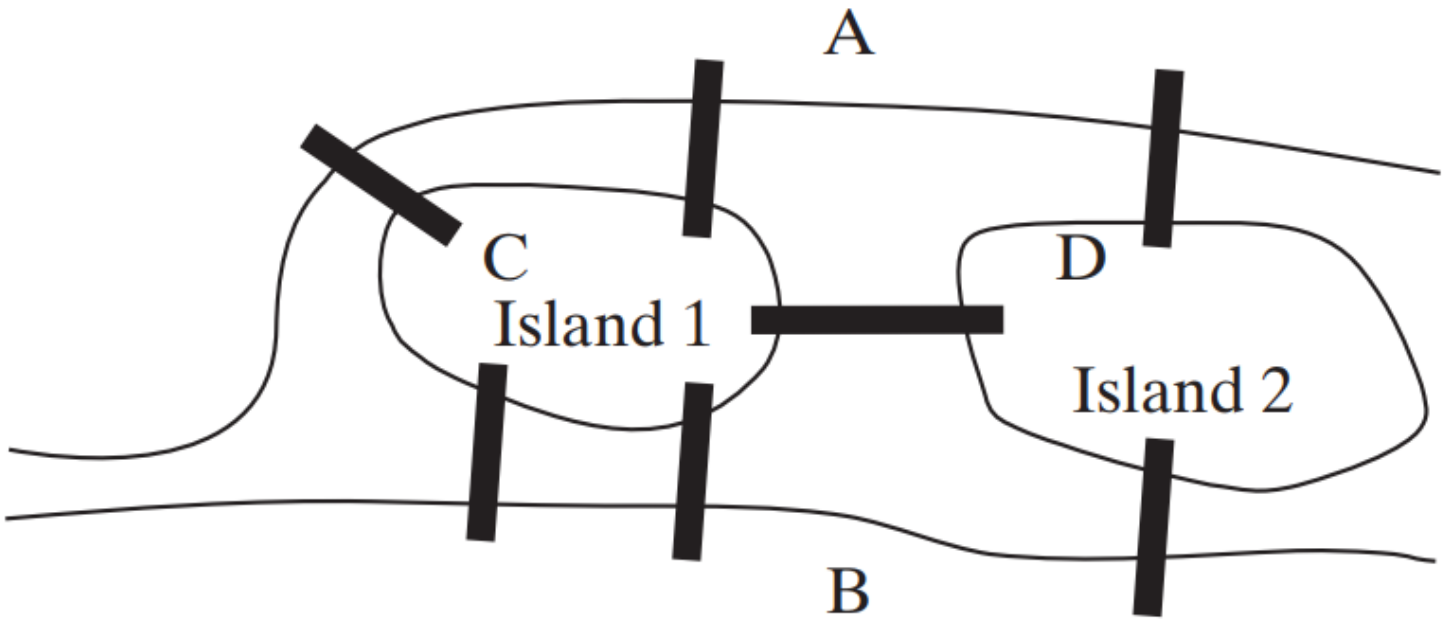
Getting Started

- Graphs are useful in modeling and solving real-world problems. For example, the problem to find the least number of flights between two cities can be modeled using a graph, where the vertices represent cities and the edges represent the flights between two adjacent cities, as shown in Figure below. The problem of finding the minimal number of connecting flights between two cities is reduced to finding a shortest path between two vertices in a graph.

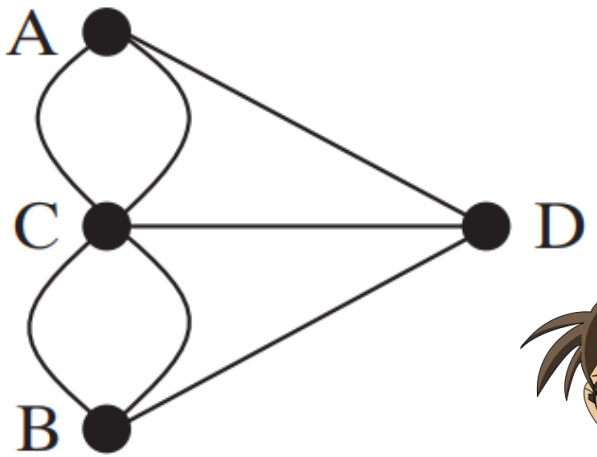


Getting Started

❖ The study of graph problems is known as *graph theory*. Graph theory was founded by Leonhard Euler in 1736, when he introduced graph terminology to solve the famous *Seven Bridges of Königsberg* problem. The city of Königsberg, Prussia (now Kaliningrad, Russia), was divided by the Pregel River. There were two islands on the river. The city and islands were connected by seven bridges, as shown in Figure a. The question is, can one take a walk, cross each bridge exactly once, and return to the starting point? Euler proved that it is not possible.



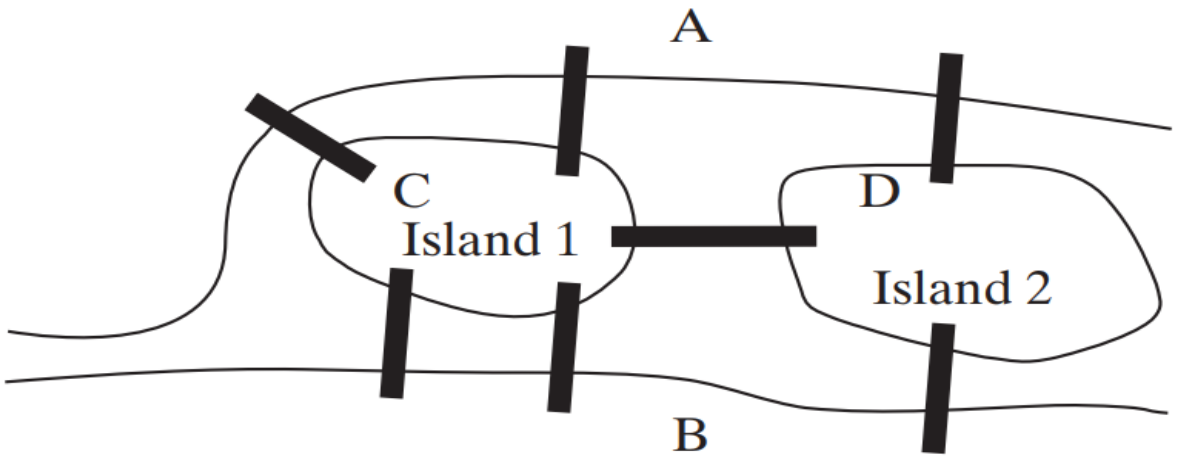
(a) Seven bridges sketch



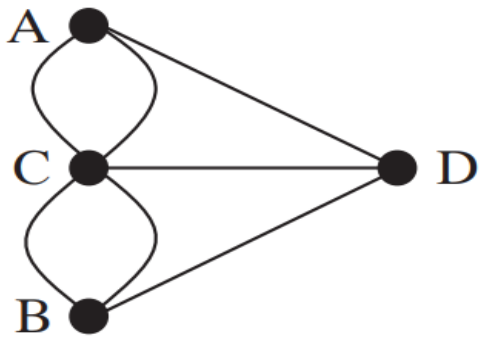
(b) Graph model

Getting Started

- To establish a proof, Euler first abstracted the Königsberg city map by eliminating all streets, producing the sketch shown in Figure a. Next, he replaced each land mass with a dot, called a *vertex* or a *node*, and each bridge with a line, called an *edge*, as shown in Figure b. This structure with vertices and edges is called a *graph*.
- Looking at the graph, we ask whether there is a path starting from any vertex, traversing all edges exactly once, and returning to the starting vertex. Euler proved that for such a path to exist, each vertex must have an even number of edges. Therefore, the Seven Bridges of Königsberg problem has no solution.



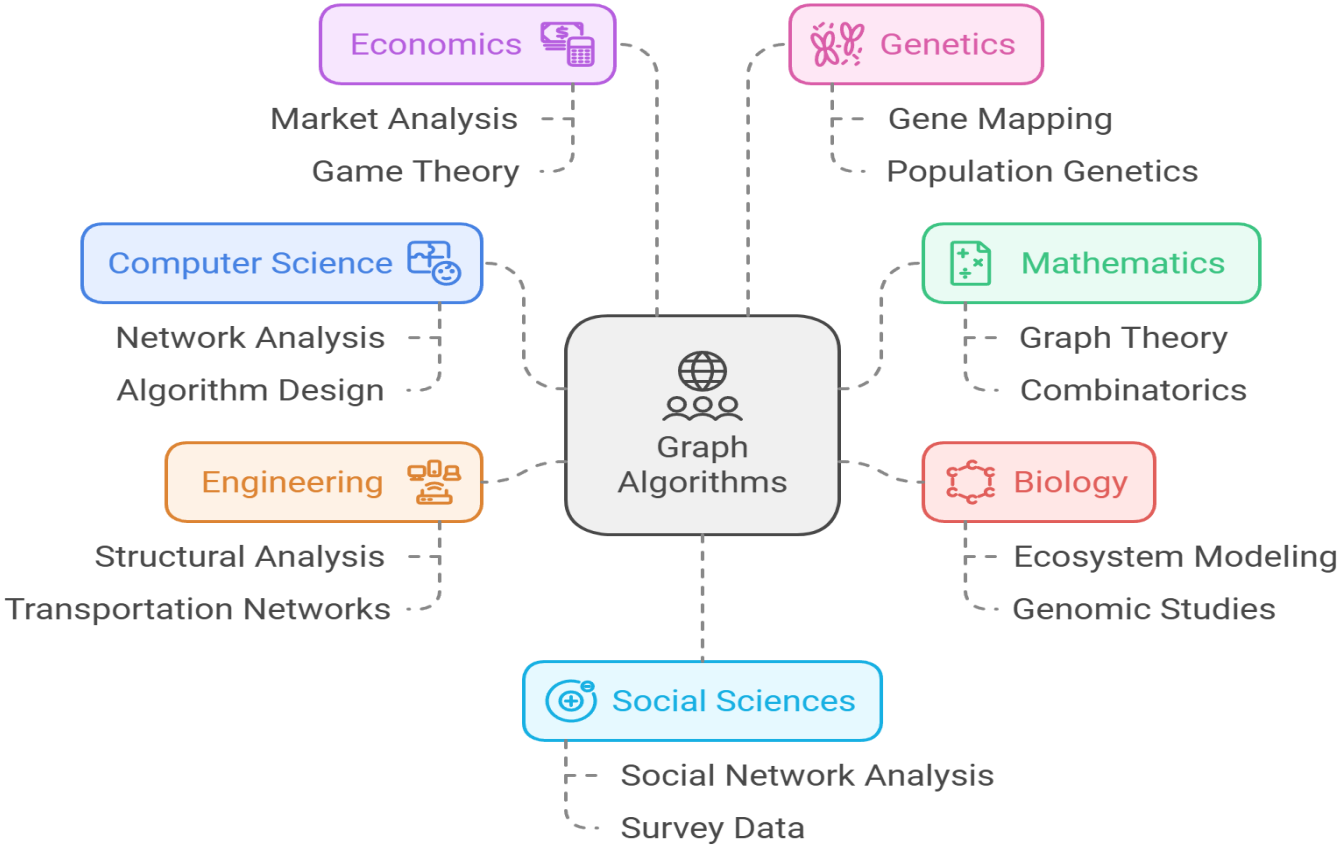
(a) Seven bridges sketch



(b) Graph model

Getting Started

❖ Graph problems are often solved using algorithms. Graph algorithms have many applications in various areas, such as in computer science, mathematics, biology, engineering, economics, genetics, and social sciences. This lecture focuses on the basics of graphs and their applications.



What You Are About To Achieve

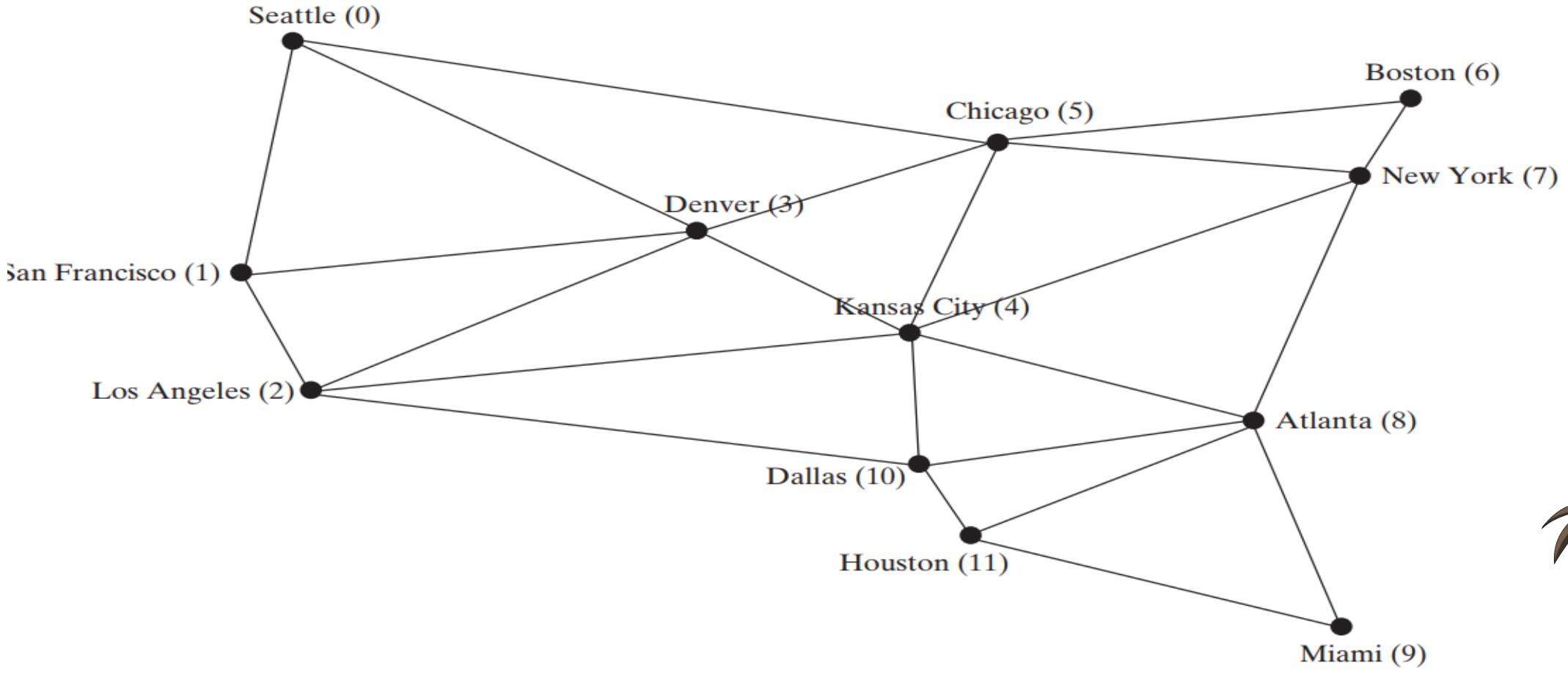
❖ By the end of this lecture, we are going to:

- ❑ **Understand** the fundamental concepts and basic terminologies related to graphs.
- ❑ **Identify** the degree of vertices in different graph structures.
- ❑ **Explore** paths, cycles, and connected graphs and their characteristics.
- ❑ **Recognize** specific types of special simple graphs and their properties.
- ❑ **Learn** different methods for representing graphs, including adjacency matrices and lists.

- **Basic Terminologies**
- The Degree of Vertices
- Path – Cycle - Connected Graph
- Special Simple Graphs
- Representing Graphs
- Final Touches

Basic Terminologies

❖ A *graph* is a mathematical structure that represents relationships among entities in the real world. For example, the graph in Figure below represents the flights among cities.



Basic Terminologies

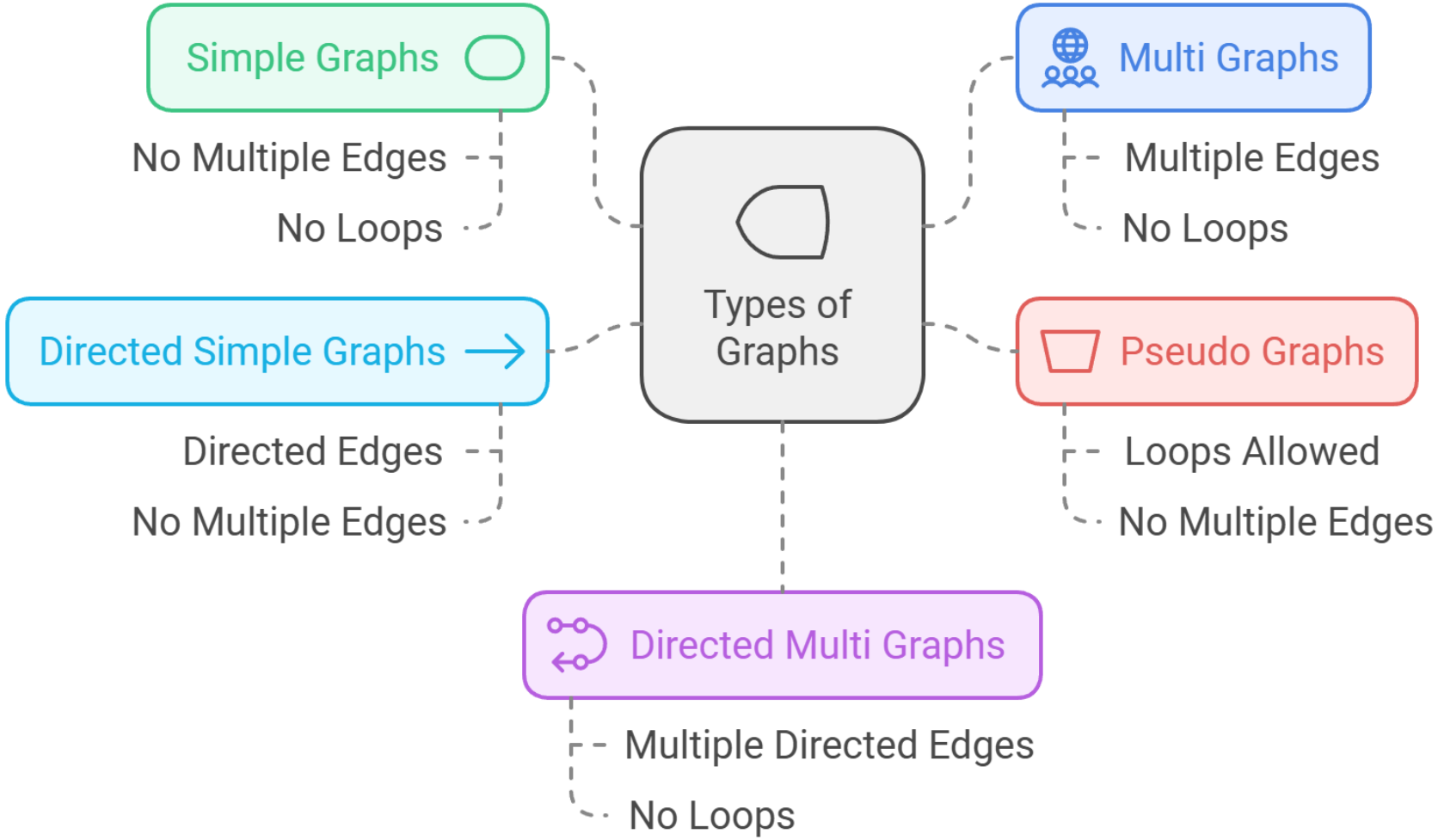
❖ A graph consists of a nonempty set of vertices (also known as *nodes* or *points*), and a set of edges that connect the vertices. For convenience, we define a graph as **G = (V, E)**, where **V** represents a set of vertices and **E** represents a set of edges. For example, **V** and **E** for the graph in the preceding Figure are as follows:

```
V = {"Seattle", "San Francisco", "Los Angeles",
     "Denver", "Kansas City", "Chicago", "Boston",
     "New York", "Atlanta", "Miami", "Dallas", "Houston"};

E = {{"Seattle", "San Francisco"}, {"Seattle", "Chicago"},
     {"Seattle", "Denver"}, {"San Francisco", "Denver"},
     ...
}
```

Basic Terminologies

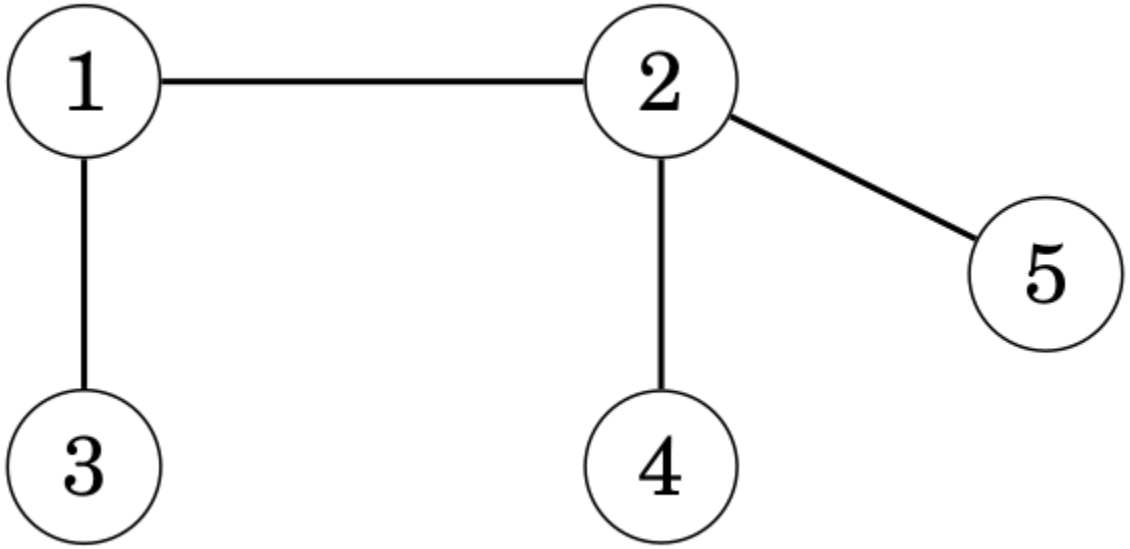
❖ Graphs are categorized based on the type and number of edges between pairs of vertices. There are five main types of graphs:



Basic Terminologies

❖ **Simple Graphs**

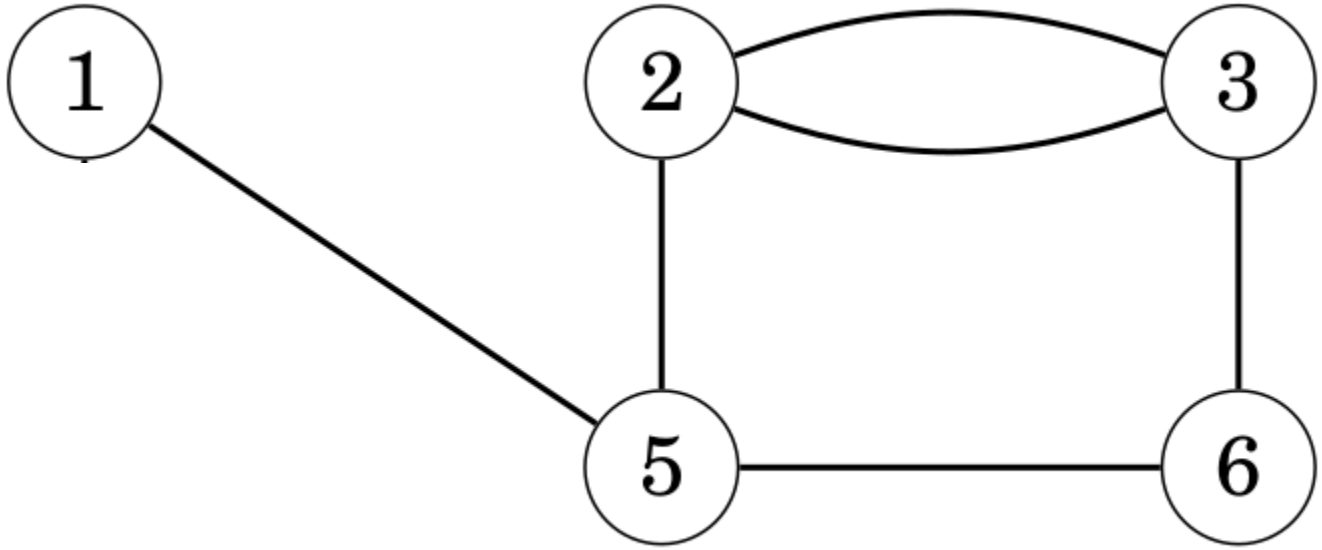
❑ An undirected simple graph **G = (V, E)** consists of **V** as a non-empty set of vertices, and **E** as a set of unordered pairs of two distinct elements of **V**, called edges.



Basic Terminologies

❖ **Multi Graphs**

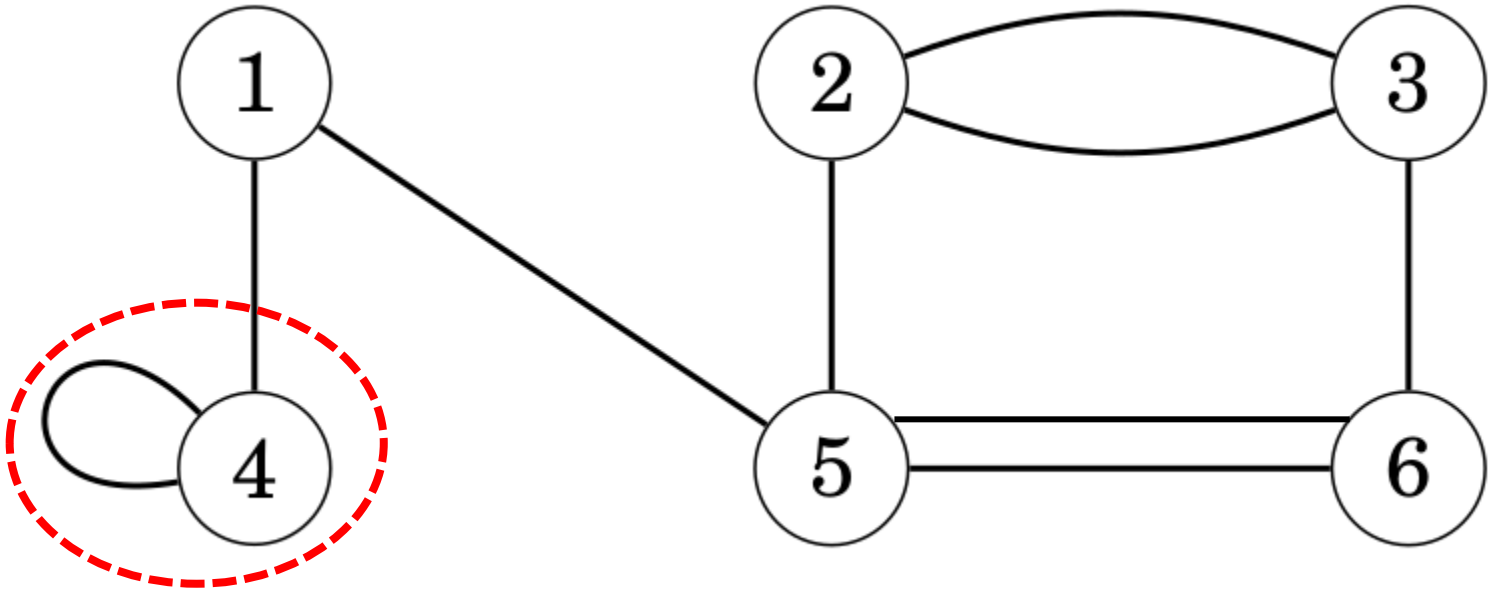
- ❑ A multigraph **G = (V, E)** consists of **V**, a non-empty set of vertices, and **E**, a set of unordered pairs of distinct vertices called edges. Two edges ***e1*** and ***e2*** are called parallel (or multiple) edges if they correspond to the same pair of vertices.
- ❑ Every simple graph is a multigraph, but not every multigraph is a simple graph, because in a multigraph, there can be two (or more) edges connecting the same pair of vertices.



Basic Terminologies

❖ **Pseudo Graphs**

- ❑ A pseudo undirected graph **G = (V, E)** consists of **V**, which is a non-empty set of vertices, and **E**, which is a set of unordered pairs of elements (not necessarily distinct) from **V**, called edges.
- ❑ For **v ∈ V**, if **(v, v) ∈ E**, then we say there is *a loop* at vertex **v**.



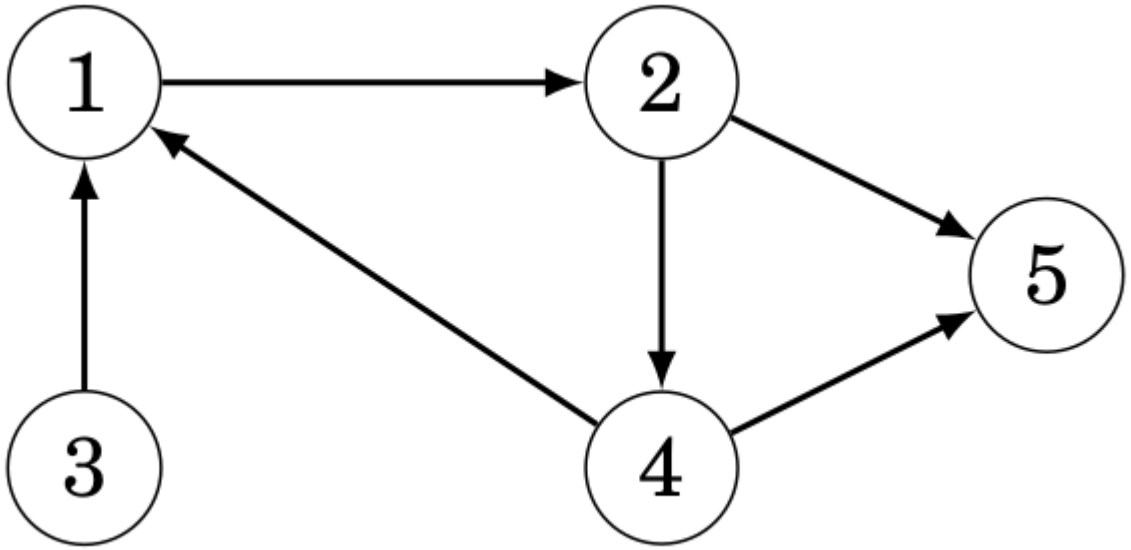
In sum,

- **A pseudo graph** is the most **general type of undirected graph** because it can contain loops and multiple edges.
- **A multigraph** is a type of undirected graph that ***can contain multiple edges but cannot have loops***.
- **A simple graph** is an undirected graph that ***does not contain multiple edges or loops***.

Basic Terminologies

❖ **Directed Simple Graphs**

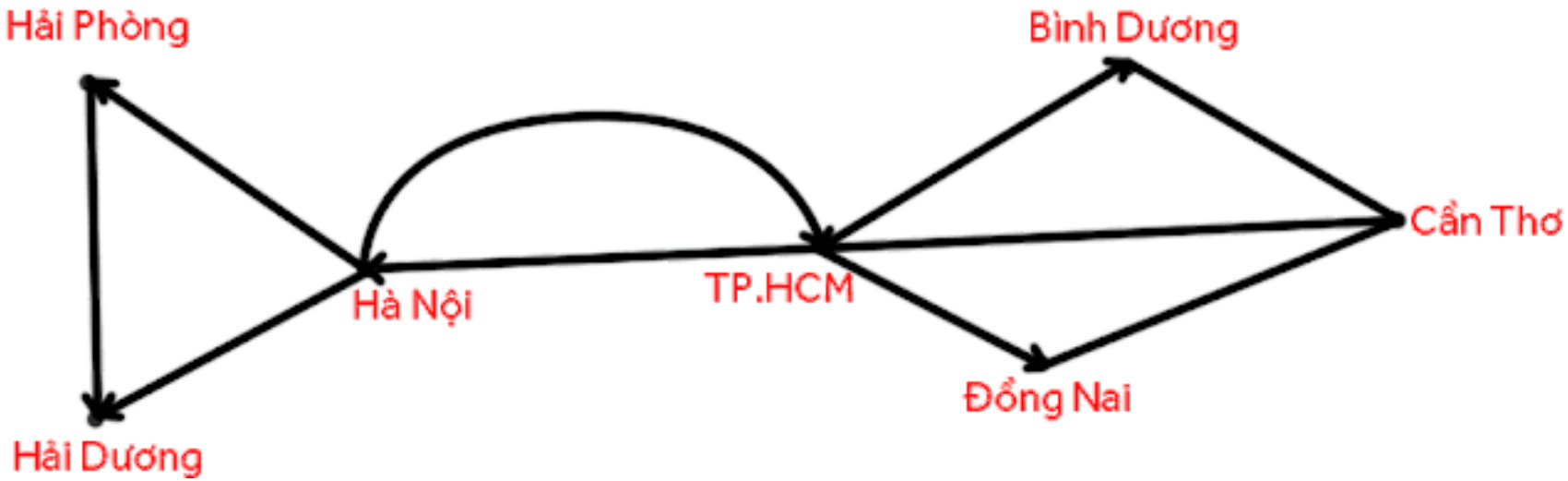
❑ The simple directed graph **G = (V, E)** consists of **V**, a non-empty set of vertices, and **E**, a set of ordered pairs of distinct elements from **V**, called edges.



Basic Terminologies

❖ **Directed Multi Graphs**

- A directed multigraph **G = (V, E)** consists of **V**, which is a non-empty set of vertices, and **E**, which is a set of ordered pairs of distinct elements from **V** called edges. Two edges ***e1*** and ***e2*** corresponding to the same pair of vertices are called parallel edges.



There were numerous type of graphs?

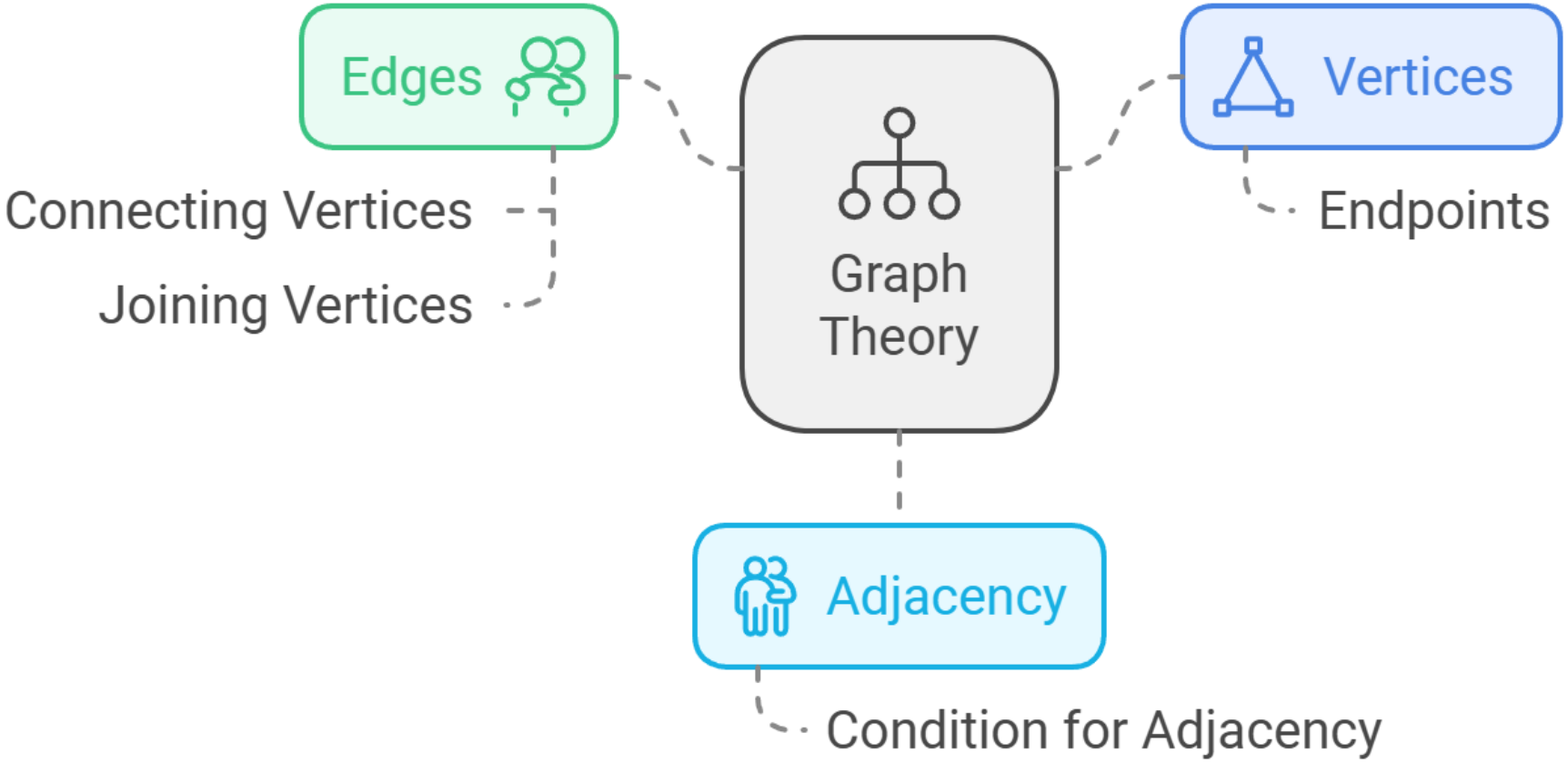
Do you have any questions?

- Basic Terminologies
- **The Degree of Vertices**
- Path – Cycle - Connected Graph
- Special Simple Graphs
- Representing Graphs
- Final Touches

The Degree of Vertices

❖ **Definition 01**

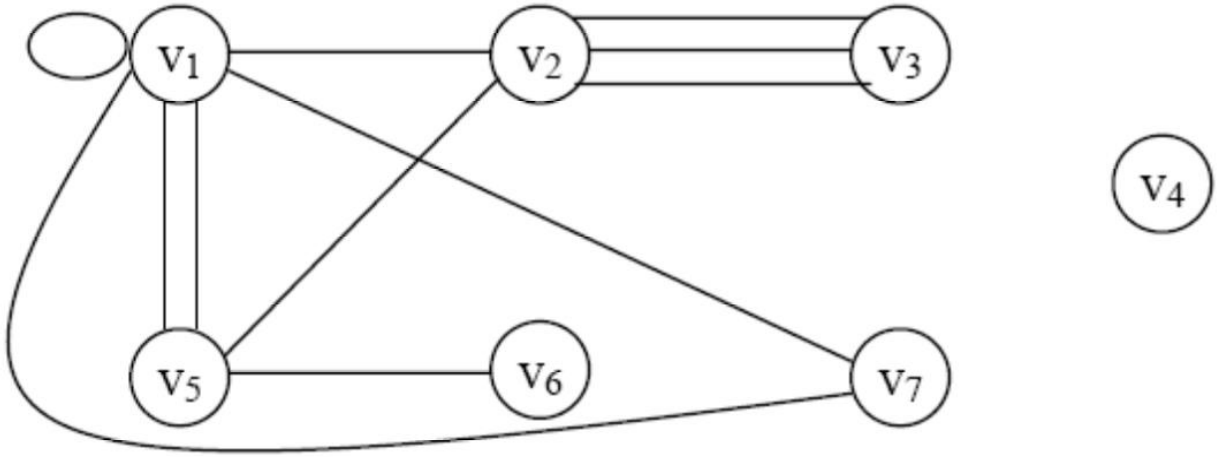
❑ Two vertices u and v in an undirected graph $G = (V,E)$ are called adjacent if $(u,v) \in E$. If $e = (u,v)$, then e is called the edge connecting vertices u and v. Edge e is also called the edge joining vertices u and v. The vertices u and v are called the endpoints of edge e.



The Degree of Vertices

❖ **Definition 02**

- ❑ The degree of a vertex **v** in a graph **G = (V, E)**, denoted as **deg(v)**, is the number of edges connected to it, where a loop at a vertex is counted twice for its degree.
- ❑ A vertex **v** is called a pendant vertex if **deg(v) = 1**, and it is called an isolated vertex if **deg(v) = 0**.



Look at the graph,
deg(v1)=7, deg(v2)=5, deg(v3)=3, deg(v4)=0, deg(v5)=4, deg(v6)=1, deg(v7)=2.
So, v4 is isolated vertex and v6 is pendant vertex.

The Degree of Vertices

❖ **Definition 03**

- ❑ Suppose **G = (V, E)** is an undirected graph with **m** edges. Then the sum of the degrees of all vertices is twice the number of edges.
- ❑ In an undirected graph, the number of vertices with **odd degrees** (i.e., vertices with odd degrees) is an **even number**.

Even

The number of vertices with odd degrees is always even in an undirected graph.

Odd

The number of vertices with odd degrees cannot be odd in an undirected graph.

The Degree of Vertices

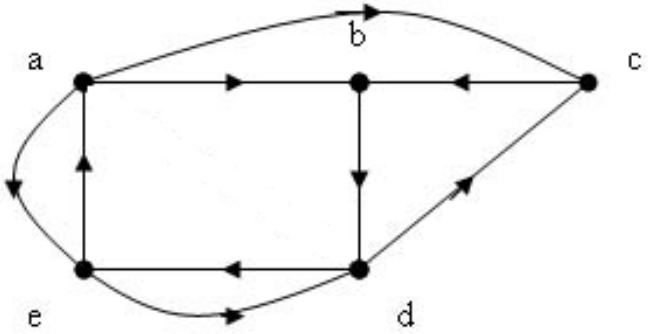
❖ **Definition 04**

❑ If **e = (u, v)** is an edge of a directed graph **G**, then we say that the vertices **u** and **v** are adjacent, and we say that the edge **(u, v)** connects vertex **u** to vertex **v** or that this edge goes out of vertex **u** and into vertex **v**. Vertex **u(v)** will be called the start (end) vertex of the edge **(u, v).**

❖ **Definition 05**

❑ We call the out-degree (in-degree) of vertex **v** in a directed graph the number of edges leaving (entering) it, and it is denoted as deg^{+}**(v)** (***deg***$^{-}$**(v)**). Then,

$$m=\sum_{v\in V}\deg^{+}(v)=\sum_{v\in V}\deg^{-}(v)$$



deg^{-}(a)=1, deg^{-}(b)=2, deg^{-}(c)=2,
deg^{-}(d)=2, deg^{-}(e) = 2.
deg^{+}(a)=3, deg^{+}(b)=1, deg^{+}(c)=1,
deg^{+}(d)=2, deg^{+}(e)=2.

Practice:
Vertices Degree

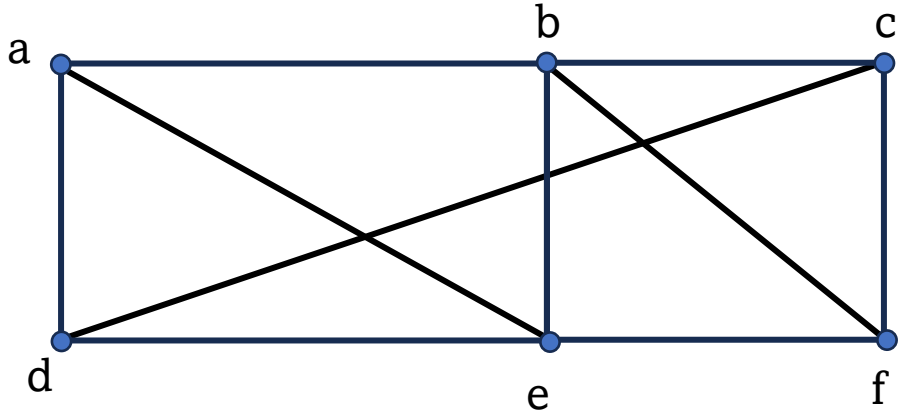
- Basic Terminologies
- The Degree of Vertices
- **Path – Cycle - Connected Graph**
- Special Simple Graphs
- Representing Graphs
- Final Touches

Path – Cycle - *Connected Graph*

- A path of length **n** from vertex **u** to vertex **v**, where **n** is a positive integer, in an undirected graph **G = (V, E)** is a sequence $\mathbf{x_0, x_1, \ldots, x_n}$, where $u = x_0$, $v = x_n$, and $(\mathbf{x_i}, \mathbf{x_{i+1}}) \in \mathbf{E}$ for $i = 0, 1, 2, \ldots, n-1$. The path can also be represented as a sequence of edges: $(x_0, x_1), (x_1, x_2), \ldots, (x_{n-1}, x_n)$.
- Vertex **u** is called the starting vertex, while vertex **v** is called the ending vertex of the path.
- A path where the starting vertex coincides with the ending vertex (i.e., u = v) is called a cycle.
- A path or cycle is called simple if no edge is repeated.

Path – Cycle - *Connected Graph*

❖ For example,

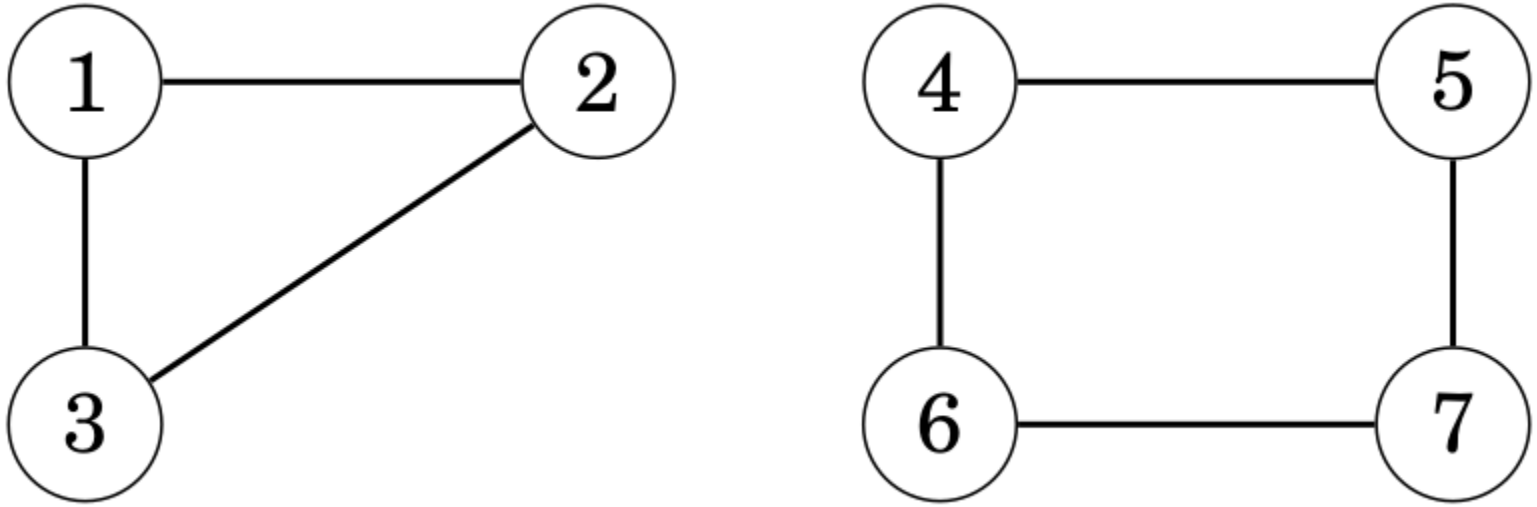


❖ On the undirected graph shown in the image:

- ❑ **a, d, c, f, e** is a simple path of ***length 4***.
- ❑ **d, e, c, a** is not a path, because (c, e) is not an edge of the graph.
- ❑ The sequence **b, c, f, e, b** is a cycle of ***length 4***.
- ❑ The path **a, b, e, d, a, b** has a length of 5 and is not a simple path, because the edge (a, b) appears twice in it.

Path – Cycle - Connected Graph

❖ The undirected graph **G = (V, E)** is called **connected** if there is always a path between any two of its vertices.

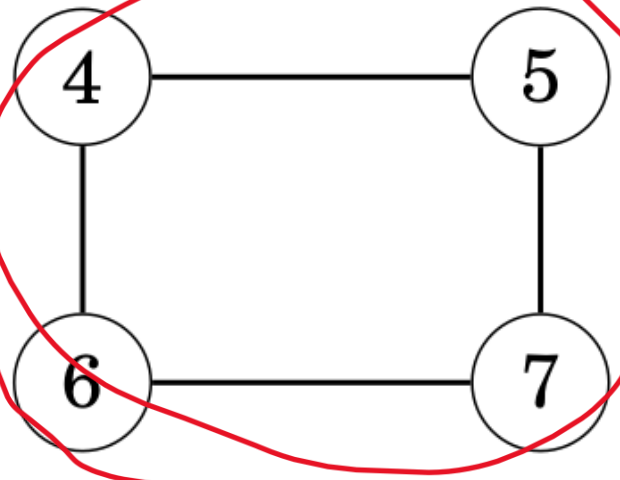


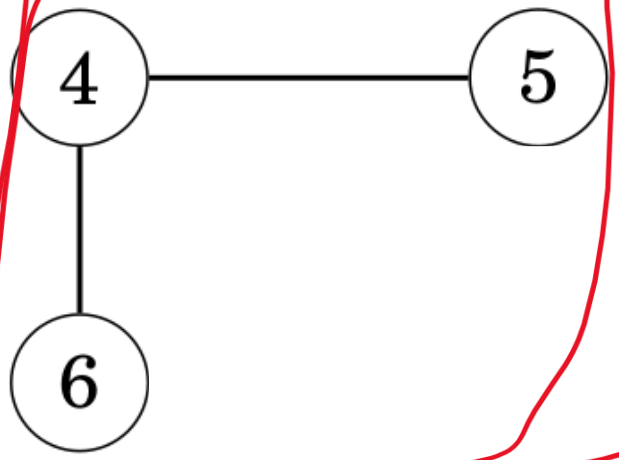
❖ **Proposition**

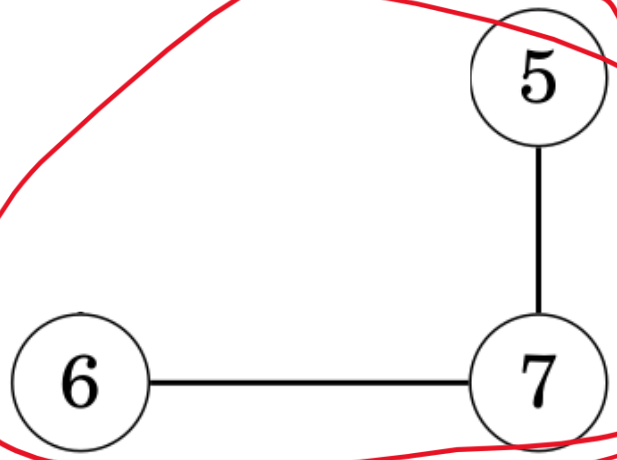
❑ **1, 2, 3** or **4, 5, 6, 7** is a **connected** graph but 1, 2, 3, 4, 5, 6, 7 is not.

Path – Cycle - Connected Graph

❖ We call a subgraph of a graph **G = (V, E)** a graph **H = (W, F)**, where **W ⊆ V** and **F ⊆ E**. In the case where the graph is disconnected, it will break down into several connected subgraphs with no common vertices. These connected subgraphs are called the connected components of the graph.





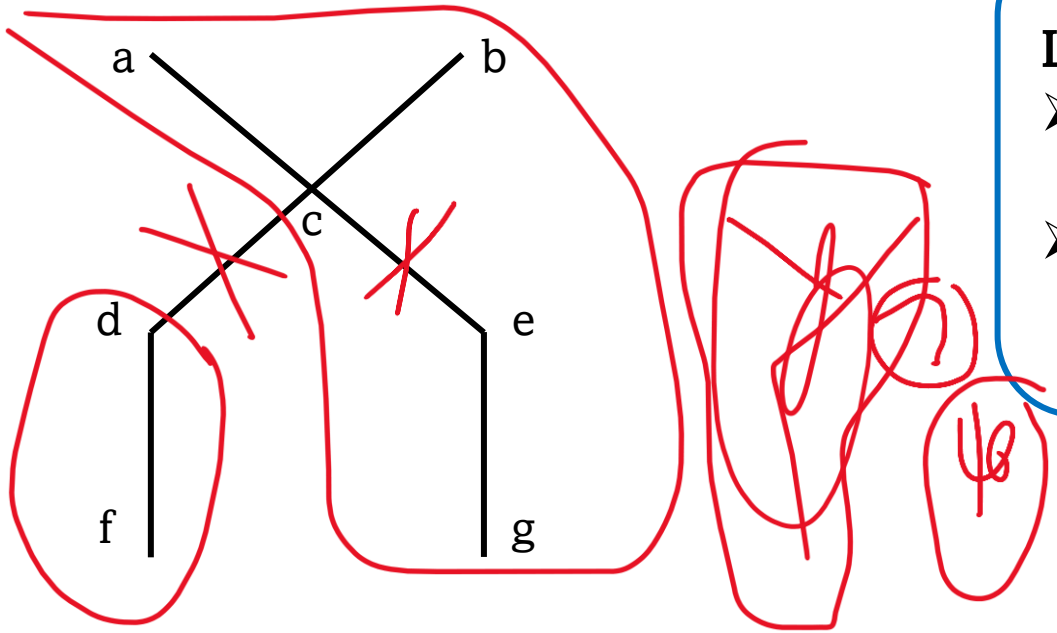


❖ **Proposition**

- ❑ **4, 5, 6, 7, 4, 5, 6, 5, 6, 7** is a **disconnected** graph but they can be broken down into several **connected** ones.

Path – Cycle - Connected Graph

- A **vertex v** is called a ***branching*** vertex if removing **v** along with the edges incident to it from the graph increases the number of connected components of the graph.
- An **edge e** is called a ***bridge*** if removing it from the graph increases the number of connected components of the graph.



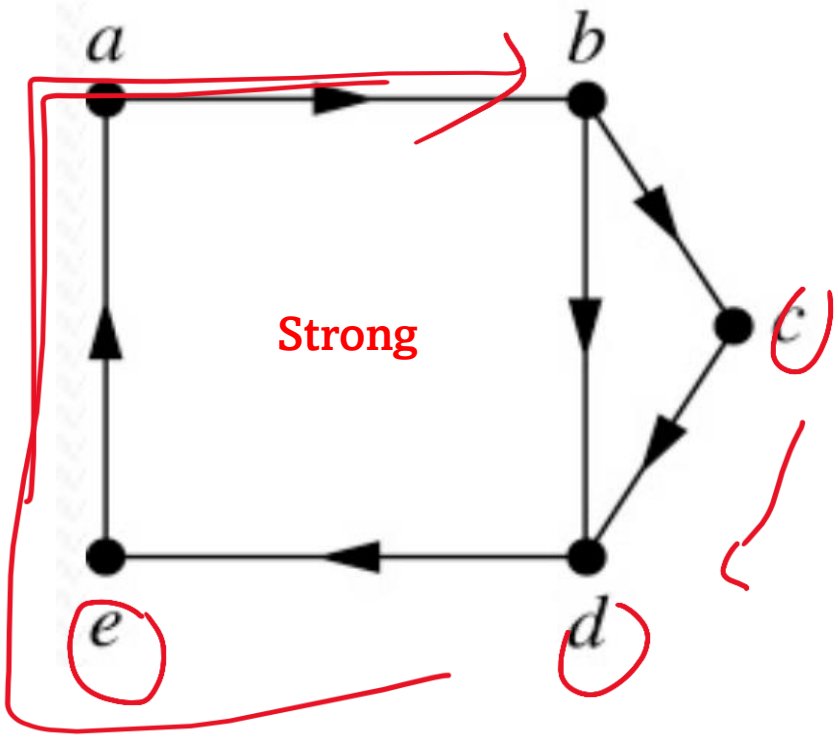
Look at the graph,

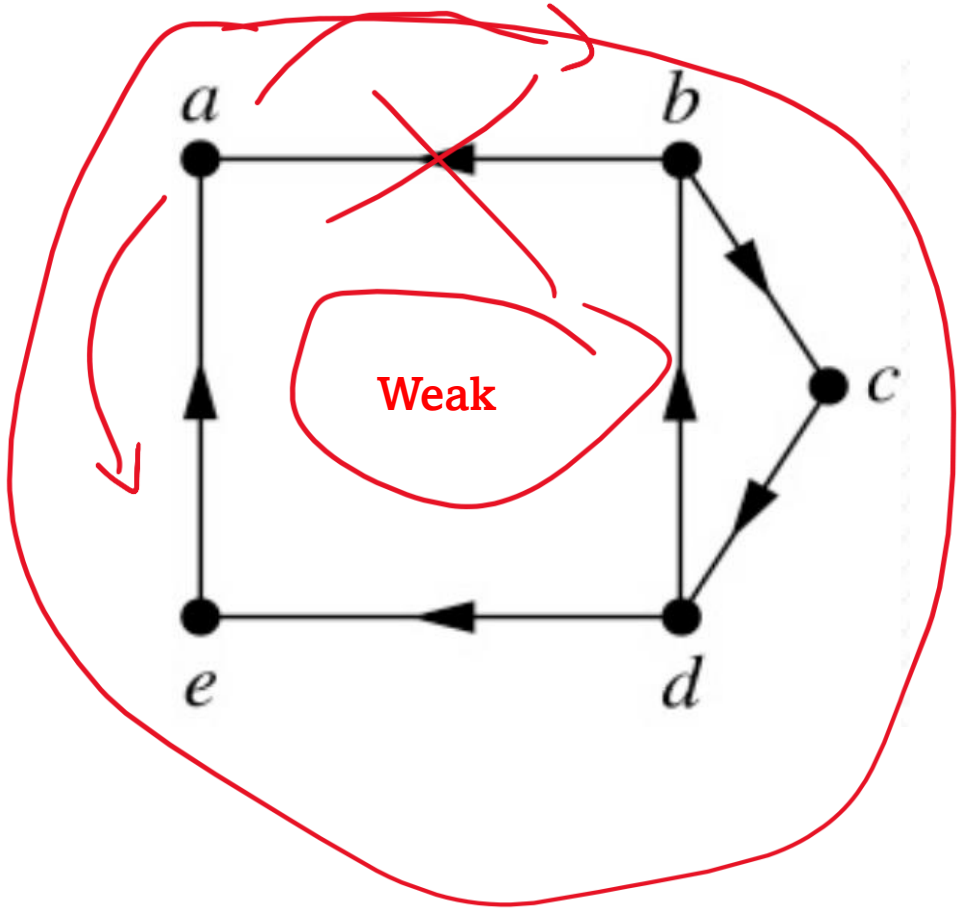
- **Vertices c, d, and e are branch vertices,**
- **while the edges (c,d) and (c,e) are bridges.**

Path – Cycle - Connected Graph

- The directed graph G = (V, A) is called **strongly connected** if there is always a path between any two vertices.
- The directed graph G = (V, A) is called **weakly connected** if its corresponding undirected graph is connected.

a → b





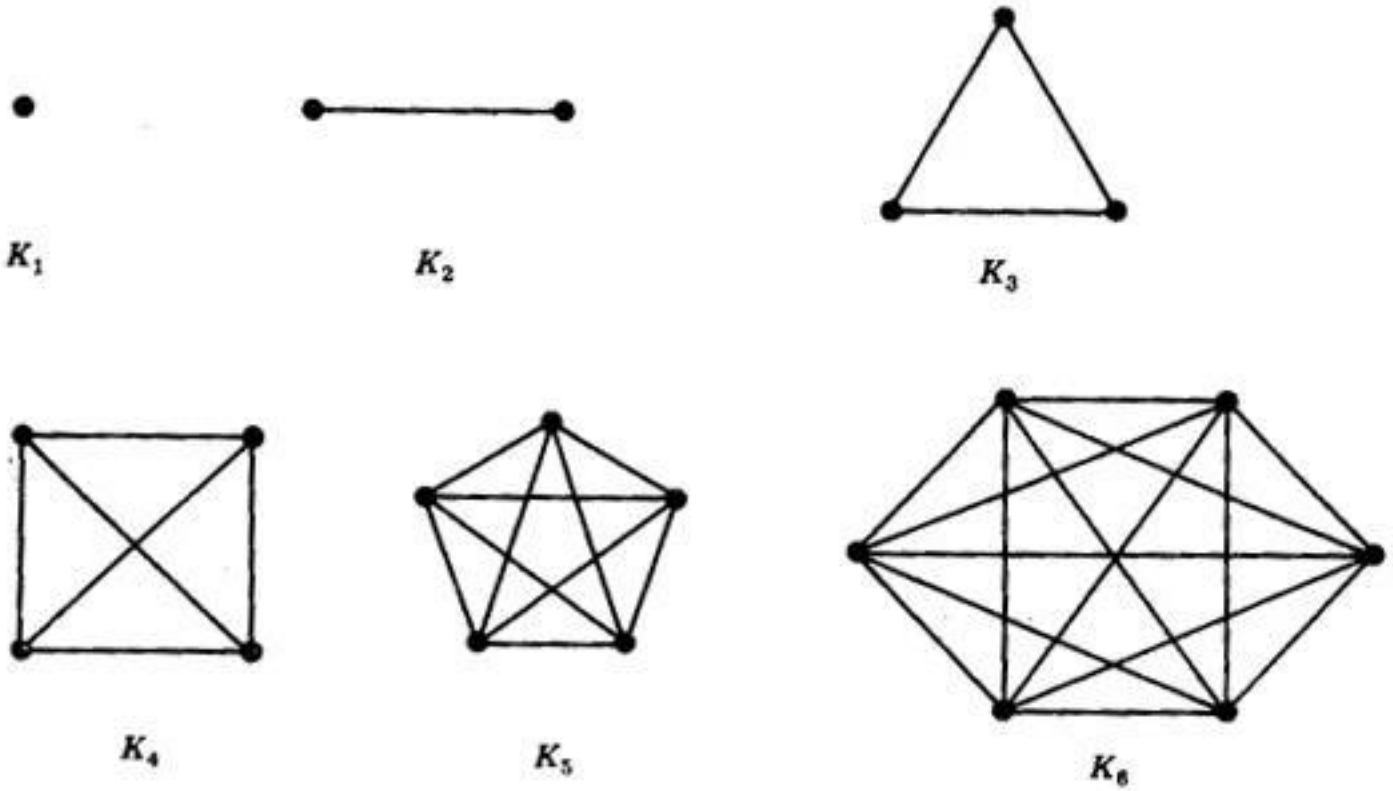
There were numerous definitions?

Do you have any questions?

- Basic Terminologies
- The Degree of Vertices
- Path – Cycle - Connected Graph
- **Special Simple Graphs**
- Representing Graphs
- Final Touches

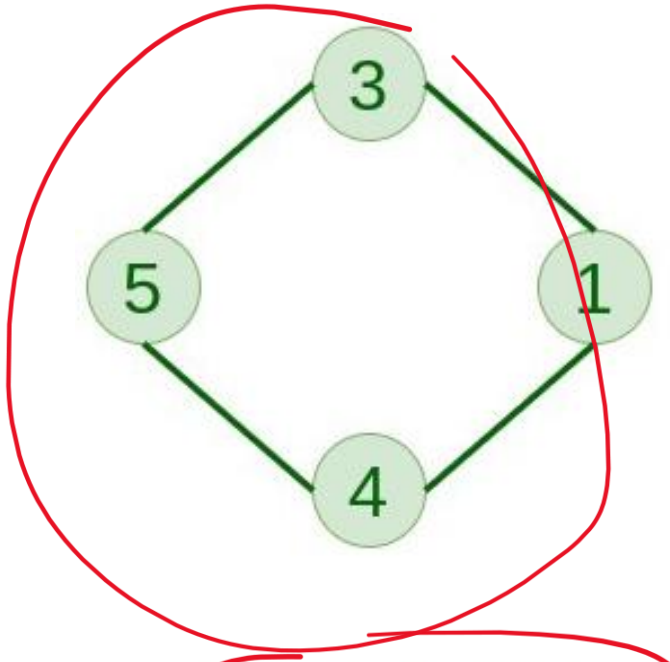
Special Simple Graphs

- **Complete graph:** A complete graph with **n** vertices, denoted as K_n, is a simple graph in which any two distinct vertices are always adjacent.
- Thus, K_n has n(n − 1)/2 edges, and each vertex of K_n has a degree of n − 1.

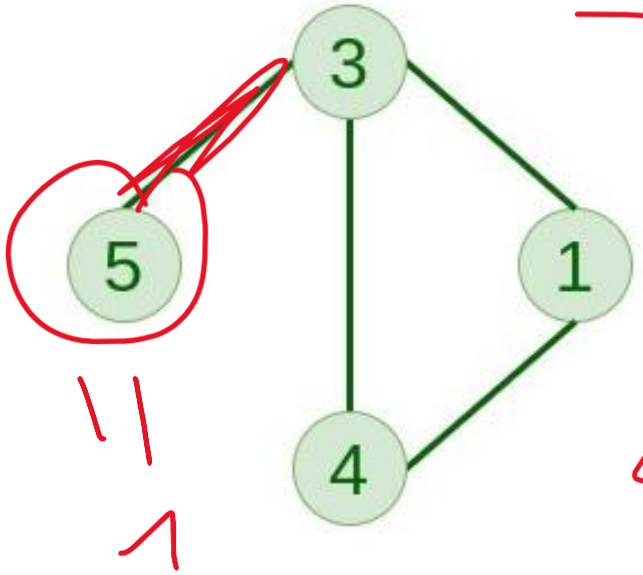


Special Simple Graphs

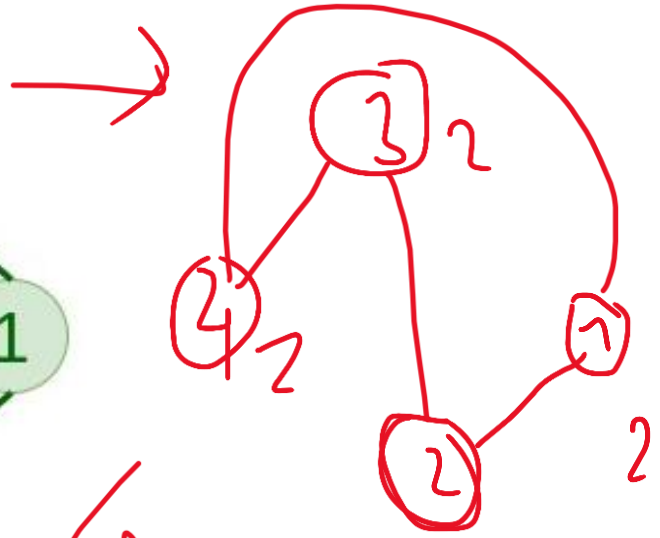
❖ **Cycle graph:** A simple graph with **n** vertices $v_1, v_2, \ldots, v_n$ (n ≥ 3) and **n** edges $(v_1, v_2), (v_2, v_3), \ldots, (v_n, v_1)$ is called a cycle graph, denoted as C_n. Thus, each vertex of C_n has degree 2.



Cycle Graph



Cyclic Graph



4

cycle 2

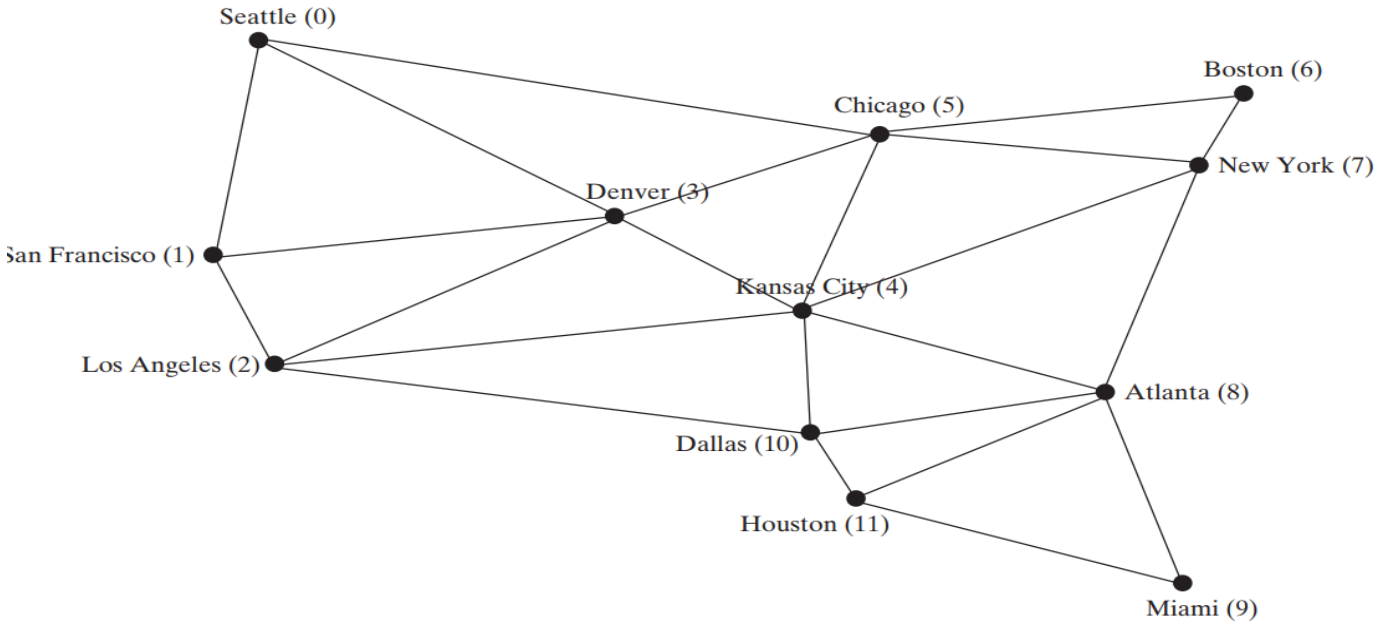
4 vertices

As we've seen, graphs come in many forms, and we've just touched the surface with cyclic and acyclic graphs. But there are many more types of graphs out there, each with unique properties and applications - such as bipartite graphs, complete graphs, and weighted graphs, to name just a few. However, due to time constraints, we'll need to move on to the next important section: How to represent a graph. Understanding how we can represent graphs efficiently is crucial because it forms the foundation for working with algorithms, whether we're solving problems or analyzing real-world networks.

- Basic Terminologies
- The Degree of Vertices
- Path – Cycle - Connected Graph
- Special Simple Graphs
- **Representing Graphs**
- Final Touches

Representing Graphs

❖ To write a program that processes and manipulates graphs, you have to store or represent data for the graphs in the computer. The vertices can be stored in an array or a list. For example, you can store all the city names in the graph using the following array:

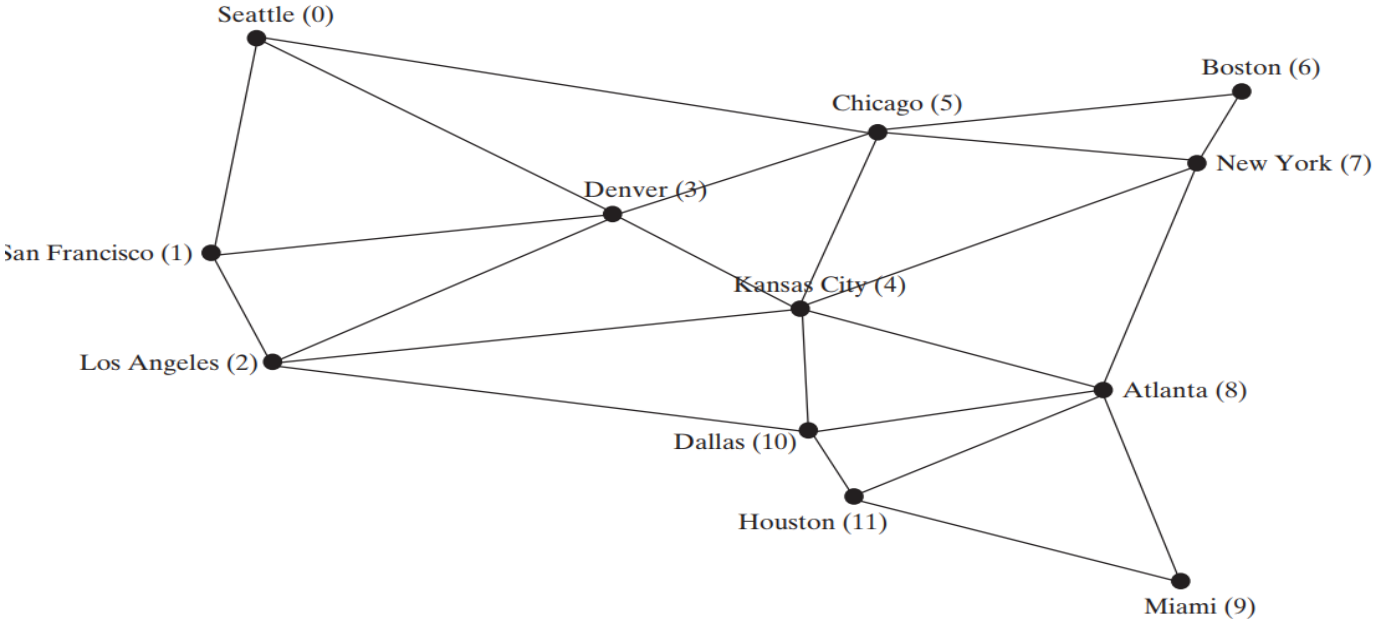


vertices[0]	Seattle
vertices[1]	San Francisco
vertices[2]	Los Angeles
vertices[3]	Denver
vertices[4]	Kansas City
vertices[5]	Chicago
vertices[6]	Boston
vertices[7]	New York
vertices[8]	Atlanta
vertices[9]	Miami
vertices[10]	Dallas
vertices[11]	Houston

```
String[] vertices = {"Seattle", "San Francisco", "Los Angeles",
"Denver", "Kansas City", "Chicago", "Boston", "New York",
"Atlanta", "Miami", "Dallas", "Houston"};
```

Representing Graphs

❖ The edges can be represented using a two-dimensional array. For example, you can store all the edges in the graph using the following array:



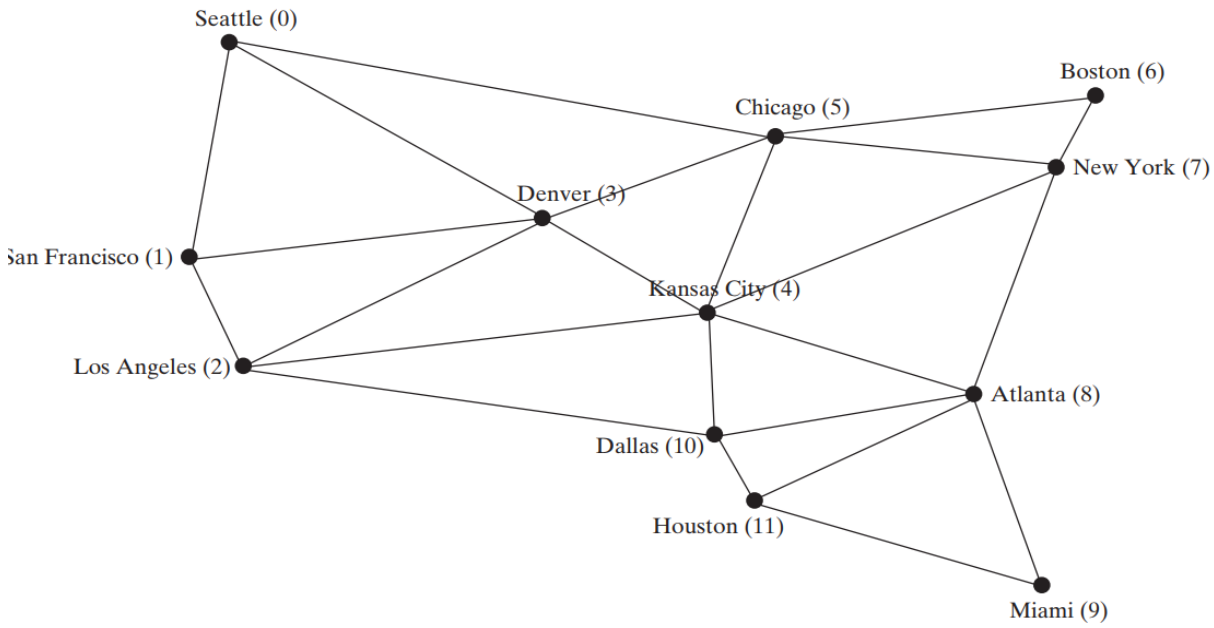

```
int[][] edges = {
        {0, 1}, {0, 3}, {0, 5},
        {1, 0}, {1, 2}, {1, 3},
        {2, 1}, {2, 3}, {2, 4}, {2, 10},
        {3, 0}, {3, 1}, {3, 2}, {3, 4}, {3, 5},
        {4, 2}, {4, 3}, {4, 5}, {4, 7}, {4, 8}, {4, 10},
        {5, 0}, {5, 3}, {5, 4}, {5, 6}, {5, 7},
        {6, 5}, {6, 7},
        {7, 4}, {7, 5}, {7, 6}, {7, 8},
        {8, 4}, {8, 7}, {8, 9}, {8, 10}, {8, 11},
        {9, 8}, {9, 11},
        {10, 2}, {10, 4}, {10, 8}, {10, 11},
        {11, 8}, {11, 9}, {11, 10}
}
```

These edge array is not efficient for internal processing?

Let’s try *adjacency matrices* and *adjacency lists*

Representing Graphs

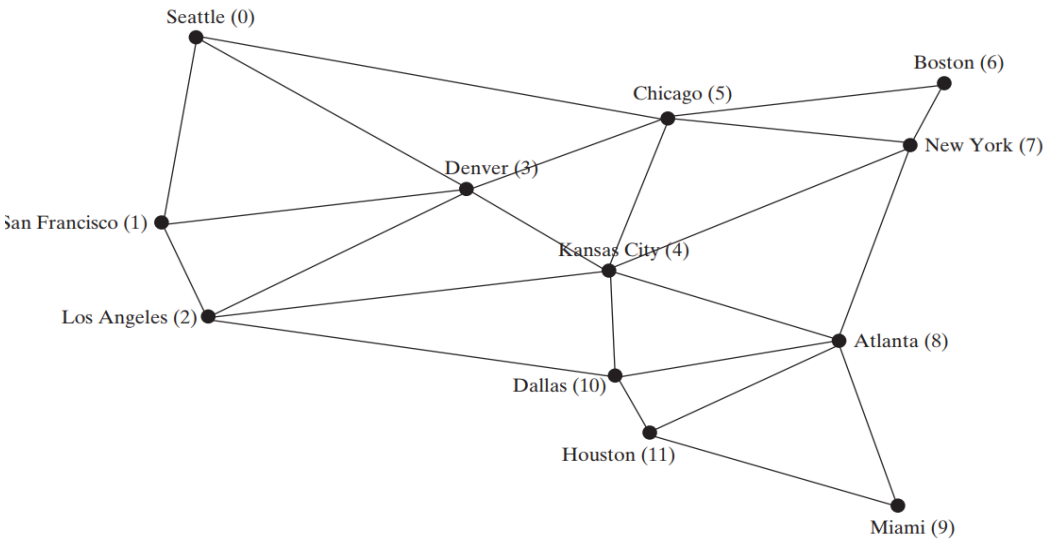
❖ Assume that the graph has *n* vertices. You can use a two-dimensional *n* * *n* matrix, say **adjacencyMatrix**, to represent the edges. Each element in the array is **0** or **1**. **adjacencyMatrix[i][j]** is **1** if there is an edge from vertex *i* to vertex *j*; otherwise, **adjacencyMatrix[i][j]** is **0**. If the graph is undirected, the matrix is symmetric, because **adjacencyMatrix[i][j]** is the same as **adjacencyMatrix[j][i]**. For example, the edges in the graph can be represented using an *adjacency matrix* as follows:




```
int[][] adjacencyMatrix = {
    {0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0}, // Seattle
    {1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0}, // San Francisco
    {0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0}, // Los Angeles
    {1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0}, // Denver
    {0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0}, // Kansas City
    {1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 0}, // Chicago
    {0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0}, // Boston
    {0, 0, 0, 0, 1, 1, 1, 0, 1, 0, 0, 0}, // New York
    {0, 0, 0, 1, 1, 0, 0, 1, 0, 1, 1, 1}, // Atlanta
    {0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1}, // Miami
    {0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1}, // Dallas
    {0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0} // Houston
}
```

Representing Graphs

❖ You can also represent edges using *adjacency vertex lists* or *adjacency edge lists*. An adjacency vertex list for a vertex ***i*** contains the vertices that are adjacent to ***i*** and an adjacency edge list for a vertex ***i*** contains the edges that are adjacent to ***i***. You may define an array of lists. The array has ***n*** entries, and each entry is a list. The adjacency vertex list for vertex ***i*** contains all the vertices ***j*** such that there is an edge from vertex ***i*** to vertex ***j***.



Seattle	`neighbors[0]`	1	3	5			
San Francisco	`neighbors[1]`	0	2	3			
Los Angeles	`neighbors[2]`	1	3	4	10		
Denver	`neighbors[3]`	0	1	2	4	5	
Kansas City	`neighbors[4]`	2	3	5	7	8	10
Chicago	`neighbors[5]`	0	3	4	6	7	
Boston	`neighbors[6]`	5	7				
New York	`neighbors[7]`	4	5	6	8		
Atlanta	`neighbors[8]`	4	7	9	10	11	
Miami	`neighbors[9]`	8	11				
Dallas	`neighbors[10]`	2	4	8	11		
Houston	`neighbors[11]`	8	9	10			

Representing Graphs

❖ Similarly, using adjacency edge list, we have:

Seattle	neighbors[0]	Edge(0, 1)	Edge(0, 3)	Edge(0, 5)			
San Francisco	neighbors[1]	Edge(1, 0)	Edge(1, 2)	Edge(1, 3)			
Los Angeles	neighbors[2]	Edge(2, 1)	Edge(2, 3)	Edge(2, 4)	Edge(2, 10)		
Denver	neighbors[3]	Edge(3, 0)	Edge(3, 1)	Edge(3, 2)	Edge(3, 4)	Edge(3, 5)	
Kansas City	neighbors[4]	Edge(4, 2)	Edge(4, 3)	Edge(4, 5)	Edge(4, 7)	Edge(4, 8)	Edge(4, 10)
Chicago	neighbors[5]	Edge(5, 0)	Edge(5, 3)	Edge(5, 4)	Edge(5, 6)	Edge(5, 7)	
Boston	neighbors[6]	Edge(6, 5)	Edge(6, 7)				
New York	neighbors[7]	Edge(7, 4)	Edge(7, 5)	Edge(7, 6)	Edge(7, 8)		
Atlanta	neighbors[8]	Edge(8, 4)	Edge(8, 7)	Edge(8, 9)	Edge(8, 10)	Edge(8, 11)	
Miami	neighbors[9]	Edge(9, 8)	Edge(9, 11)				
Dallas	neighbors[10]	Edge(10, 2)	Edge(10, 4)	Edge(10, 8)	Edge(10, 11)		
Houston	neighbors[11]	Edge(11, 8)	Edge(11, 9)	Edge(11, 10)			

For further details, please refer to the **additional reading materials** available on the **LMS**.

Now that you have learned Graph and Its Application. Do you have any question? If you don’t, in the next section, I will show the summary of this lecture…

Summary

1. A *graph* is a useful mathematical structure that represents relationships among entities in the real world. You learned how to model graphs using classes and interfaces, how to represent vertices and edges using arrays and linked lists, and how to implement operations for graphs.

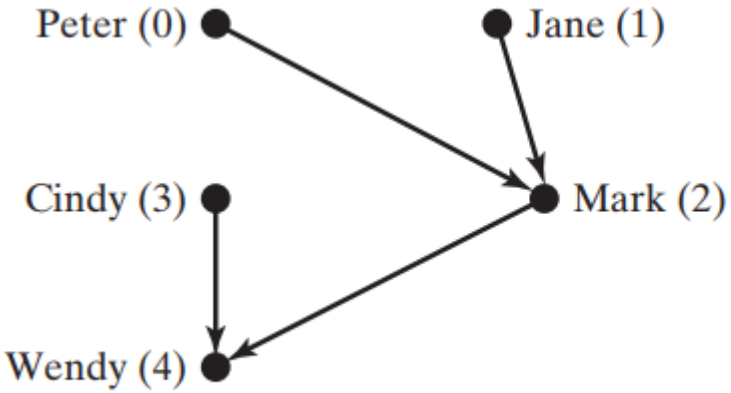
2. Graph traversal is the process of visiting each vertex in the graph exactly once. You learned two popular ways for traversing a graph: the *depth-first search* (DFS) and *breadth-first search* (BFS).

3. DFS and BFS can be used to solve many problems such as detecting whether a graph is connected, detecting whether there is a cycle in the graph, and finding a shortest path between two vertices.

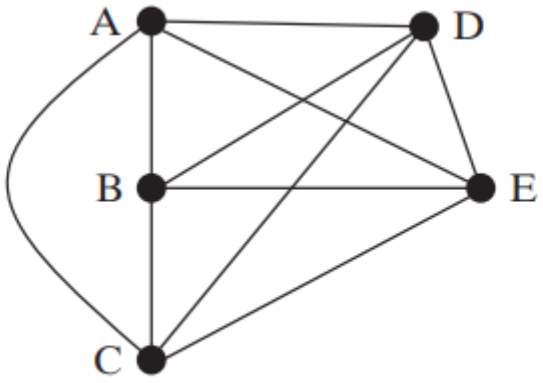
This brings us to the end of this lecture! It's time for Final Touches…

Final Touches

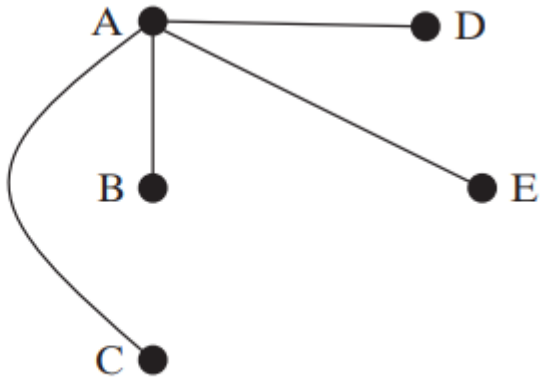
- A graph may be directed or undirected. In a *directed graph*, each edge has a direction, which indicates that you can move from one vertex to the other through the edge. You can model parent/child relationships using a directed graph, where an edge from vertex A to B indicates that A is a parent of B. Figure below shows a directed graph.



(a) A directed graph



(b) A complete graph



(c) A subgraph of the graph in (b)

Final Touches

- In an *undirected graph*, you can move in both directions between vertices. Edges may be weighted or unweighted. For example, you can assign a weight for each edge in the graph in the preceding Figure to indicate the flight time between the two cities.
- Two vertices in a graph are said to be *adjacent* if they are connected by the same edge. Similarly, two edges are said to be *adjacent* if they are connected to the same vertex. An edge in a graph that joins two vertices is said to be *incident* to both vertices. The *degree* of a vertex is the number of edges incident to it.
- Two vertices are called *neighbors* if they are adjacent. Similarly, two edges are called *neighbors* if they are adjacent.
- A *loop* is an edge that links a vertex to itself. If two vertices are connected by two or more edges, these edges are called *parallel edges*. A *simple graph* is one that has doesn't have any loops or parallel edges. In a *complete graph*, every two pairs of vertices are connected, as shown in the preceding Figure b.
- A graph is *connected* if there exists a path between any two vertices in the graph. A *subgraph* of a graph *G* is a graph whose vertex set is a subset of that of *G* and whose edge set is a subset of that of *G*. For example, the graph in the preceding Figure c is a subgraph of the graph in the preceding Figure b.
- Assume that the graph is connected and undirected. A *cycle* is a closed path that starts from a vertex and ends at the same vertex. A connected graph is a *tree* if it does not have cycles. A *spanning tree* of a graph *G* is a connected subgraph of *G* and the subgraph is a tree that contains all vertices in G.

Thanks!

Any questions?

For an in-depth understanding of Java, I highly recommend referring to the textbooks. This slide provides a brief overview and may not cover all the details you're eager to explore!